

Chapter 37 — String Dynamic Programming

0. What You Will Learn in This Chapter

String Dynamic Programming is one of the most important DP categories for high-level coding interviews.

String DP is used when a problem involves:

- two strings
- prefixes of strings
- subsequences
- edit operations
- palindromes
- pattern matching
- counting distinct subsequences
- segmenting strings
- matching wildcards
- longest common subsequence
- longest palindromic subsequence

Most string DP problems use states like:

```
dp[i]
dp[i][j]
dp[l][r]
```

By the end of this chapter, you should understand:

- How to define DP over string prefixes
- How to define DP over substring ranges
- How to solve two-string DP
- How to solve subsequence DP
- How to solve palindrome DP
- How to solve edit-operation DP
- How to solve pattern-matching DP
- How to handle empty prefixes
- How to initialize first row/column
- How to distinguish substring vs subsequence
- How to avoid off-by-one mistakes
- How to optimize some string DP problems

Core problems covered in this chapter:

1. Longest Common Subsequence
2. Edit Distance
3. Delete Operation for Two Strings
4. Distinct Subsequences
5. Interleaving String
6. Regular Expression Matching
7. Wildcard Matching
8. Longest Palindromic Subsequence
9. Longest Palindromic Substring
10. Palindromic Substrings
11. Palindrome Partitioning II

12. Word Break
 13. Decode Ways
 14. Minimum ASCII Delete Sum for Two Strings
 15. Shortest Common Supersequence
 16. Scramble String, preview
 17. Count Different Palindromic Subsequences, advanced preview
-

1. What Is String DP?

String DP solves problems where the state depends on positions inside one or more strings.

Common state styles:

1.1 Prefix DP

$dp[i]$ = answer for prefix $s[0..i-1]$

Example:

- Word Break
- Decode Ways

1.2 Two-String Prefix DP

$dp[i][j]$ = answer for $s1[0..i-1]$ and $s2[0..j-1]$

Example:

- Longest Common Subsequence
- Edit Distance

1.3 Substring Range DP

$dp[l][r]$ = answer for substring $s[l..r]$

Example:

- Longest Palindromic Subsequence
 - Palindromic Substrings
 - Palindrome Partitioning
-

2. Why String DP Exists

Brute force string problems often try many possibilities:

- delete or keep characters
- match or skip characters
- split a string
- choose subsequence characters
- compare all prefixes
- try all edit operations
- try all palindrome cuts

These choices overlap.

Example:

LCS of:

abcde
ace

If characters do not match, we try:

LCS(i - 1, j)

LCS(i, j - 1)

The same (i, j) states repeat often.

DP stores those states.

3. Core String DP Idea

For strings, always ask:

What prefixes/substrings am I comparing?

For two-string DP:

$dp[i][j]$ = answer using first i chars of string A and first j chars of string B

This avoids negative indices and makes empty-prefix base cases easy.

Character positions are usually:

$A[i - 1]$

$B[j - 1]$

when filling $dp[i][j]$.

4. Substring vs Subsequence

This distinction is critical.

4.1 Substring

A substring is contiguous.

Example:

"abcde"

Substrings:

"abc", "bcd", "cde"

Problems:

- Longest Palindromic Substring
- Palindromic Substrings
- Minimum Window Substring, not DP usually

4.2 Subsequence

A subsequence preserves order but does not need to be contiguous.

Example:

"abcde"

Subsequence:

"ace"

Problems:

- Longest Common Subsequence
- Longest Palindromic Subsequence

-
- Distinct Subsequences
-

5. When to Use String DP

Use string DP when the problem asks for:

5.1 Longest Common / Matching Subsequence

Examples:

- LCS
- Delete Operation for Two Strings
- Minimum ASCII Delete Sum

5.2 Edit Operations

Examples:

- Edit Distance
- Delete Operation
- Minimum Insertions to Palindrome

5.3 Count Subsequences

Examples:

- Distinct Subsequences

5.4 Pattern Matching

Examples:

- Regular Expression Matching
- Wildcard Matching

5.5 Palindromes

Examples:

- Longest Palindromic Subsequence
- Palindromic Substrings
- Palindrome Partitioning II

5.6 String Segmentation

Examples:

- Word Break
-

6. When NOT to Use String DP

6.1 Simple Frequency Problem

Use hashing/counting.

Example:

Valid Anagram.

6.2 Simple Substring Window

Use sliding window.

Example:

Longest Substring Without Repeating Characters.

6.3 Exact Pattern Search

Use KMP, rolling hash, or built-in search.

6.4 Generate All Partitions

Use backtracking.

Example:

Palindrome Partitioning I returns all partitions.

6.5 Need Lexicographically Smallest Greedy Result

May use stack/greedy.

Example:

Remove Duplicate Letters.

7. Mental Models

7.1 Prefix Grid

For two strings, imagine a grid.

Rows are prefixes of string A.

Columns are prefixes of string B.

Each cell answers:

What is the answer for these two prefixes?

7.2 Match or Skip

Many string DP transitions are:

If characters match:

use both

If not:

skip one side

7.3 Empty Prefix

$dp[0][j]$ means first string is empty.

$dp[i][0]$ means second string is empty.

This often defines base cases.

7.4 Palindrome Shrinking

For palindrome range DP:

$dp[l][r]$

depends on:

$dp[l + 1][r - 1]$

because palindrome properties move inward.

8. Pattern Recognition Checklist

Ask:

18. Does the problem involve one or two strings?
19. Are we comparing prefixes?
20. Are we matching subsequences?
21. Are edit operations allowed?
22. Does the problem ask longest/common/min operations?
23. Does it involve palindromes?
24. Does it involve string segmentation?
25. Does brute force try skip/delete/replace choices?

- 26. Can state be `dp[i][j]`?
 - 27. Can state be `dp[l][r]`?
-

9. C++ Tools Needed

9.1 2D DP Table

```
vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
```

9.2 Boolean DP Table

```
vector<vector<bool>> dp(n, vector<bool>(n, false));
```

9.3 String Indexing

If using `dp[i][j]` for first `i` and first `j` chars:

```
char a = word1[i - 1];  
char b = word2[j - 1];
```

9.4 Substring

```
s.substr(start, length)
```

Be careful: substring creation costs time.

10. Core Templates

Template 1 — Two-String Prefix DP

```
vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));  
  
for (int i = 1; i <= m; i++) {  
    for (int j = 1; j <= n; j++) {  
        if (a[i - 1] == b[j - 1]) {  
            dp[i][j] = matchTransition;  
        } else {  
            dp[i][j] = nonMatchTransition;  
        }  
    }  
}
```

Template 2 — LCS

```
if (a[i - 1] == b[j - 1]) {  
    dp[i][j] = 1 + dp[i - 1][j - 1];  
} else {  
    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);  
}
```

Template 3 — Edit Distance

```
if (a[i - 1] == b[j - 1]) {
    dp[i][j] = dp[i - 1][j - 1];
} else {
    dp[i][j] = 1 + min({
        dp[i - 1][j],    // delete
        dp[i][j - 1],    // insert
        dp[i - 1][j - 1] // replace
    });
}
```

Template 4 — Palindrome Range DP

```
for (int len = 1; len <= n; len++) {
    for (int l = 0; l + len - 1 < n; l++) {
        int r = l + len - 1;

        if (s[l] == s[r]) {
            dp[l][r] = ...
        } else {
            dp[l][r] = ...
        }
    }
}
```

Template 5 — Word Break

```
dp[0] = true;

for (int i = 1; i <= n; i++) {
    for (int j = 0; j < i; j++) {
        if (dp[j] && dict.count(s.substr(j, i - j))) {
            dp[i] = true;
            break;
        }
    }
}
```

11. Major String DP Variations

Two-string prefix	$dp[i][j]$	LCS
Edit operations	$dp[i][j]$	Edit Distance
Count subsequences	$dp[i][j]$	Distinct Subsequences
Interleaving	$dp[i][j]$	Interleaving String
Pattern matching	$dp[i][j]$	Regex / Wildcard
Palindrome range	$dp[l][r]$	LPS
Palindrome substring	$dp[l][r]$ boolean	Palindromic Substrings
Prefix segmentation	$dp[i]$	Word Break
Decode count	$dp[i]$	Decode Ways

12. Full Problem Walkthrough 1 — LeetCode 1143: Longest Common Subsequence

12.1 Problem Summary

Given two strings, return length of their longest common subsequence.

Example:

```
text1 = "abcde"
text2 = "ace"
```

Output:

3

because "ace" is common.

12.2 How to Identify String DP

Need compare two strings and choose subsequence characters.

State should use two prefixes.

12.3 State

$dp[i][j]$ = LCS length of $text1[0..i-1]$ and $text2[0..j-1]$

12.4 Transition

If last characters match:

$dp[i][j] = 1 + dp[i - 1][j - 1]$

Else skip one character:

```
dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
```

12.5 C++ Code

```
class Solution {
public:
    int longestCommonSubsequence(string text1, string text2) {
        int m = text1.size();
        int n = text2.size();

        vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (text1[i - 1] == text2[j - 1]) {
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                } else {
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }

        return dp[m][n];
    }
};
```

12.6 Dry Run Concept

For:

```
abcde
ace
```

Matches:

```
a, c, e
```

DP grows from prefix matches.

12.7 Complexity

Time:

```
O(m * n)
```

Space:

```
O(m * n)
```

13. Full Problem Walkthrough 2 —

LeetCode 72: Edit Distance

13.1 Problem Summary

Given word1 and word2, return minimum operations to convert word1 into word2.

Allowed operations:

- insert
- delete
- replace

13.2 How to Identify String DP

Two strings.

Need minimum edit operations.

Use prefix DP.

13.3 State

$dp[i][j]$ = min operations to convert first i chars of word1
into first j chars of word2

13.4 Base Cases

Convert prefix to empty:

$dp[i][0] = i$

Convert empty to prefix:

$dp[0][j] = j$

13.5 Transition

If characters match:

$dp[i][j] = dp[i - 1][j - 1]$

Else:

delete: $dp[i - 1][j]$

insert: $dp[i][j - 1]$

replace: $dp[i - 1][j - 1]$

Add 1 operation.

13.6 C++ Code

```
class Solution {
public:
    int minDistance(string word1, string word2) {
        int m = word1.size();
        int n = word2.size();

        vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

        for (int i = 0; i <= m; i++) {
```

```

        dp[i][0] = i;
    }

    for (int j = 0; j <= n; j++) {
        dp[0][j] = j;
    }

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1[i - 1] == word2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1];
            } else {
                dp[i][j] = 1 + min({
                    dp[i - 1][j],
                    dp[i][j - 1],
                    dp[i - 1][j - 1]
                });
            }
        }
    }

    return dp[m][n];
}
};

```

13.7 Complexity

Time:

$O(m * n)$

Space:

$O(m * n)$

14. Full Problem Walkthrough 3 — LeetCode 583: Delete Operation for Two Strings

14.1 Problem Summary

Given two strings, return minimum deletions needed to make them equal.

14.2 Approach 1 — LCS

If longest common subsequence length is lcs , then delete everything else.

Answer:

```
word1.size() - lcs + word2.size() - lcs
```

14.3 C++ Code

```
class Solution {
public:
    int minDistance(string word1, string word2) {
        int m = word1.size();
        int n = word2.size();

        vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1[i - 1] == word2[j - 1]) {
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                } else {
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }

        int lcs = dp[m][n];

        return (m - lcs) + (n - lcs);
    }
};
```

14.4 Complexity

Time:

$O(m * n)$

Space:

$O(m * n)$

15. Full Problem Walkthrough 4 — LeetCode 115: Distinct Subsequences

15.1 Problem Summary

Given strings s and t .

Return number of distinct subsequences of s equal to t .

Example:

```
s = "rabbbit"
t = "rabbit"
```

Output:

3

15.2 How to Identify String DP

Need count ways to match one string as subsequence of another.

Use prefix DP.

15.3 State

$dp[i][j]$ = number of ways first i chars of s can form first j chars of t

15.4 Base Cases

Empty t can be formed by deleting all chars of s :

$dp[i][0] = 1$

Non-empty t cannot be formed from empty s :

$dp[0][j] = 0$

15.5 Transition

If characters match:

$dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j]$

Meaning:

- use current character
- skip current character

If not match:

$dp[i][j] = dp[i - 1][j]$

15.6 C++ Code

```
class Solution {
public:
    int numDistinct(string s, string t) {
        int m = s.size();
        int n = t.size();

        vector<vector<double>> dp(m + 1, vector<double>(n + 1, 0));

        for (int i = 0; i <= m; i++) {
            dp[i][0] = 1;
        }

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
```

```

        if (s[i - 1] == t[j - 1]) {
            dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];
        } else {
            dp[i][j] = dp[i - 1][j];
        }
    }
}

return (int)dp[m][n];
}
};

```

15.7 Complexity

Time:

$O(m * n)$

Space:

$O(m * n)$

15.8 Note

Use a larger numeric type because counts may become large internally.

16. Full Problem Walkthrough 5 — LeetCode 97: Interleaving String

16.1 Problem Summary

Given strings $s1$, $s2$, and $s3$.

Return true if $s3$ can be formed by interleaving $s1$ and $s2$.

16.2 State

$dp[i][j]$ = true if $s3[0..i+j-1]$ can be formed from
 $s1[0..i-1]$ and $s2[0..j-1]$

16.3 Transition

If next char comes from $s1$:

$dp[i][j] = dp[i - 1][j] \ \&\& \ s1[i - 1] == s3[i + j - 1]$

If next char comes from $s2$:

$dp[i][j] = dp[i][j - 1] \ \&\& \ s2[j - 1] == s3[i + j - 1]$

16.4 C++ Code

```

class Solution {
public:
    bool isInterleave(string s1, string s2, string s3) {

```

```

int m = s1.size();
int n = s2.size();

if (m + n != s3.size()) {
    return false;
}

vector<vector<bool>> dp(m + 1, vector<bool>(n + 1, false));

dp[0][0] = true;

for (int i = 0; i <= m; i++) {
    for (int j = 0; j <= n; j++) {
        if (i > 0 && dp[i - 1][j] && s1[i - 1] == s3[i + j - 1]) {
            dp[i][j] = true;
        }

        if (j > 0 && dp[i][j - 1] && s2[j - 1] == s3[i + j - 1]) {
            dp[i][j] = true;
        }
    }
}

return dp[m][n];
};

```

16.5 Complexity

Time:

$O(m * n)$

Space:

$O(m * n)$

17. Full Problem Walkthrough 6 — LeetCode 10: Regular Expression Matching

17.1 Problem Summary

Given string s and pattern p .

Pattern supports:

. matches any single character
* matches zero or more of previous element

Return whether pattern matches entire string.

17.2 State

$dp[i][j] = \text{true}$ if $s[0..i-1]$ matches $p[0..j-1]$

17.3 Transition

If pattern char is normal or .:

$dp[i][j] = dp[i - 1][j - 1]$ if chars match

If pattern char is *:

* can represent zero of previous char:

$dp[i][j] = dp[i][j - 2]$

Or one/more if previous pattern char matches current string char:

$dp[i][j] |= dp[i - 1][j]$

17.4 C++ Code

```
class Solution {
public:
    bool isMatch(string s, string p) {
        int m = s.size();
        int n = p.size();

        vector<vector<bool>> dp(m + 1, vector<bool>(n + 1, false));

        dp[0][0] = true;

        for (int j = 2; j <= n; j++) {
            if (p[j - 1] == '*') {
                dp[0][j] = dp[0][j - 2];
            }
        }

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (p[j - 1] == '.' || p[j - 1] == s[i - 1]) {
                    dp[i][j] = dp[i - 1][j - 1];
                } else if (p[j - 1] == '*') {
                    dp[i][j] = dp[i][j - 2];

                    if (p[j - 2] == '.' || p[j - 2] == s[i - 1]) {
```

```

        dp[i][j] = dp[i][j] || dp[i - 1][j];
    }
}
}
}

return dp[m][n];
}
};

```

17.5 Complexity

Time:

$O(m * n)$

Space:

$O(m * n)$

18. Full Problem Walkthrough 7 — LeetCode 44: Wildcard Matching

18.1 Problem Summary

Pattern supports:

? matches any single character

* matches any sequence of characters, including empty

Return true if pattern matches whole string.

18.2 State

$dp[i][j]$ = true if $s[0..i-1]$ matches $p[0..j-1]$

18.3 Transition

If char matches or ?:

$dp[i][j] = dp[i - 1][j - 1]$

If pattern char is *:

* can match empty:

$dp[i][j - 1]$

or consume one char:

$dp[i - 1][j]$

So:

$dp[i][j] = dp[i][j - 1] || dp[i - 1][j]$

18.4 C++ Code

```

class Solution {
public:
    bool isMatch(string s, string p) {
        int m = s.size();
        int n = p.size();

        vector<vector<bool>> dp(m + 1, vector<bool>(n + 1, false));

        dp[0][0] = true;

        for (int j = 1; j <= n; j++) {
            if (p[j - 1] == '*') {
                dp[0][j] = dp[0][j - 1];
            }
        }

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (p[j - 1] == '?' || p[j - 1] == s[i - 1]) {
                    dp[i][j] = dp[i - 1][j - 1];
                } else if (p[j - 1] == '*') {
                    dp[i][j] = dp[i][j - 1] || dp[i - 1][j];
                }
            }
        }

        return dp[m][n];
    }
};

```

18.5 Complexity

Time:

$O(m * n)$

Space:

$O(m * n)$

19. Full Problem Walkthrough 8 — LeetCode 516: Longest Palindromic Subsequence

19.1 Problem Summary

Given string, return length of longest palindromic subsequence.

Example:

s = "bbbab"

Output:

4

because "bbbb".

19.2 How to Identify String DP

Palindrome over subsequence.

Need range state.

19.3 State

dp[l][r] = length of longest palindromic subsequence inside s[l..r]

19.4 Transition

If chars match:

dp[l][r] = 2 + dp[l + 1][r - 1]

If not:

dp[l][r] = max(dp[l + 1][r], dp[l][r - 1])

19.5 C++ Code

```
class Solution {
public:
    int longestPalindromeSubseq(string s) {
        int n = s.size();

        vector<vector<int>> dp(n, vector<int>(n, 0));

        for (int i = 0; i < n; i++) {
            dp[i][i] = 1;
        }

        for (int len = 2; len <= n; len++) {
            for (int l = 0; l + len - 1 < n; l++) {
                int r = l + len - 1;

                if (s[l] == s[r]) {
                    dp[l][r] = 2 + dp[l + 1][r - 1];
                } else {
                    dp[l][r] = max(dp[l + 1][r], dp[l][r - 1]);
                }
            }
        }
    }
};
```

```
    }  
  
    return dp[0][n - 1];  
}  
};
```

19.6 Complexity

Time:

$O(n^2)$

Space:

$O(n^2)$

20. Full Problem Walkthrough 9 — LeetCode 5: Longest Palindromic Substring

20.1 Problem Summary

Return longest palindromic substring.
Substring must be contiguous.

20.2 DP State

$dp[l][r] = \text{true}$ if $s[l..r]$ is palindrome

20.3 Transition

$s[l] == s[r]$ and inner substring is palindrome

For length ≤ 2 , matching ends are enough.

20.4 C++ Code

```
class Solution {  
public:  
    string longestPalindrome(string s) {  
        int n = s.size();  
  
        vector<vector<bool>> dp(n, vector<bool>(n, false));  
  
        int bestStart = 0;  
        int bestLen = 1;  
  
        for (int len = 1; len <= n; len++) {  
            for (int l = 0; l + len - 1 < n; l++) {  
                int r = l + len - 1;
```

```

        if (s[l] == s[r]) {
            if (len <= 2 || dp[l + 1][r - 1]) {
                dp[l][r] = true;

                if (len > bestLen) {
                    bestLen = len;
                    bestStart = l;
                }
            }
        }
    }

    return s.substr(bestStart, bestLen);
}
};

```

20.5 Complexity

Time:

$O(n^2)$

Space:

$O(n^2)$

20.6 Note

Expand-around-center solves this in $O(1)$ space.

21. Full Problem Walkthrough 10 — LeetCode 647: Palindromic Substrings

21.1 Problem Summary

Count all palindromic substrings.

21.2 DP State

$dp[l][r] = \text{true}$ if $s[l..r]$ is palindrome

Every true state contributes 1.

21.3 C++ Code

```

class Solution {
public:
    int countSubstrings(string s) {

```

```

int n = s.size();

vector<vector<bool>> dp(n, vector<bool>(n, false));

int count = 0;

for (int len = 1; len <= n; len++) {
    for (int l = 0; l + len - 1 < n; l++) {
        int r = l + len - 1;

        if (s[l] == s[r]) {
            if (len <= 2 || dp[l + 1][r - 1]) {
                dp[l][r] = true;
                count++;
            }
        }
    }
}

return count;
};

```

21.4 Complexity

Time:

$O(n^2)$

Space:

$O(n^2)$

22. Full Problem Walkthrough 11 — LeetCode 132: Palindrome Partitioning II

22.1 Problem Summary

Given string, partition it so every substring is palindrome.
Return minimum cuts needed.

Example:

`s = "aab"`

Output:

1

because:

"aa" | "b"

22.2 State

First precompute palindrome table:

isPal[l][r]

Then:

dp[i] = minimum cuts needed for s[0..i]

22.3 Transition

If s[0..i] is palindrome:

dp[i] = 0

Else:

dp[i] = min(dp[j] + 1)
where s[j+1..i] is palindrome

22.4 C++ Code

```
class Solution {
public:
    int minCut(string s) {
        int n = s.size();

        vector<vector<bool>> isPal(n, vector<bool>(n, false));

        for (int len = 1; len <= n; len++) {
            for (int l = 0; l + len - 1 < n; l++) {
                int r = l + len - 1;

                if (s[l] == s[r]) {
                    if (len <= 2 || isPal[l + 1][r - 1]) {
                        isPal[l][r] = true;
                    }
                }
            }
        }

        vector<int> dp(n, 0);

        for (int i = 0; i < n; i++) {
            if (isPal[0][i]) {
```

```

        dp[i] = 0;
    } else {
        dp[i] = i;

        for (int j = 0; j < i; j++) {
            if (isPal[j + 1][i]) {
                dp[i] = min(dp[i], dp[j] + 1);
            }
        }
    }
}

return dp[n - 1];
}
};

```

22.5 Complexity

Time:

$O(n^2)$

Space:

$O(n^2)$

23. Full Problem Walkthrough 12 — LeetCode 139: Word Break

23.1 Problem Summary

Given string and dictionary, determine if string can be segmented into dictionary words.

23.2 State

$dp[i]$ = true if prefix $s[0..i-1]$ can be segmented

23.3 C++ Code

```

class Solution {
public:
    bool wordBreak(string s, vector<string>& wordDict) {
        unordered_set<string> dict(wordDict.begin(), wordDict.end());

        int n = s.size();

        vector<bool> dp(n + 1, false);
        dp[0] = true;
    }
};

```

```

    for (int i = 1; i <= n; i++) {
        for (int j = 0; j < i; j++) {
            if (dp[j] && dict.count(s.substr(j, i - j))) {
                dp[i] = true;
                break;
            }
        }
    }

    return dp[n];
}
};

```

23.4 Complexity

Simple version:

$O(n^3)$

because substring creation costs time.

24. Full Problem Walkthrough 13 — LeetCode 91: Decode Ways

24.1 Problem Summary

Count ways to decode digit string.

24.2 State

$dp[i]$ = number of ways to decode first i chars

24.3 C++ Code

```

class Solution {
public:
    int numDecodings(string s) {
        int n = s.size();

        vector<int> dp(n + 1, 0);
        dp[0] = 1;

        for (int i = 1; i <= n; i++) {
            if (s[i - 1] != '0') {
                dp[i] += dp[i - 1];
            }
        }
    }
}

```

```

        if (i >= 2) {
            int value = (s[i - 2] - '0') * 10 + (s[i - 1] - '0');

            if (value >= 10 && value <= 26) {
                dp[i] += dp[i - 2];
            }
        }
    }

    return dp[n];
}
};

```

25. Full Problem Walkthrough 14 — LeetCode 712: Minimum ASCII Delete Sum for Two Strings

25.1 Problem Summary

Delete characters from two strings so they become equal. Return minimum total ASCII value of deleted characters.

25.2 State

$dp[i][j]$ = minimum ASCII delete sum to make $s1[0..i-1]$ and $s2[0..j-1]$ equal

25.3 Transition

If chars match:

$dp[i][j] = dp[i - 1][j - 1]$

Else delete one char:

$dp[i][j] = \min(\begin{aligned} &dp[i - 1][j] + \text{ASCII}(s1[i - 1]), \\ &dp[i][j - 1] + \text{ASCII}(s2[j - 1]) \end{aligned})$

25.4 C++ Code

```

class Solution {
public:
    int minimumDeleteSum(string s1, string s2) {
        int m = s1.size();
        int n = s2.size();
    }
};

```

```

vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

for (int i = 1; i <= m; i++) {
    dp[i][0] = dp[i - 1][0] + s1[i - 1];
}

for (int j = 1; j <= n; j++) {
    dp[0][j] = dp[0][j - 1] + s2[j - 1];
}

for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
        if (s1[i - 1] == s2[j - 1]) {
            dp[i][j] = dp[i - 1][j - 1];
        } else {
            dp[i][j] = min(
                dp[i - 1][j] + s1[i - 1],
                dp[i][j - 1] + s2[j - 1]
            );
        }
    }
}

return dp[m][n];
};

```

25.5 Complexity

Time:

$O(m * n)$

Space:

$O(m * n)$

26. Full Problem Walkthrough 15 — LeetCode 1092: Shortest Common Supersequence

26.1 Problem Summary

Return shortest string that has both str1 and str2 as subsequences.

26.2 Key Idea

Use LCS.

Common characters should appear once.

Non-common characters must be included.

26.3 C++ Code

```
class Solution {
public:
    string shortestCommonSupersequence(string str1, string str2) {
        int m = str1.size();
        int n = str2.size();

        vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (str1[i - 1] == str2[j - 1]) {
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                } else {
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
                }
            }
        }

        string result;

        int i = m;
        int j = n;

        while (i > 0 && j > 0) {
            if (str1[i - 1] == str2[j - 1]) {
                result.push_back(str1[i - 1]);
                i--;
                j--;
            } else if (dp[i - 1][j] > dp[i][j - 1]) {
                result.push_back(str1[i - 1]);
                i--;
            } else {
                result.push_back(str2[j - 1]);
                j--;
            }
        }

        while (i > 0) {
```

```

        result.push_back(str1[i - 1]);
        i--;
    }

    while (j > 0) {
        result.push_back(str2[j - 1]);
        j--;
    }

    reverse(result.begin(), result.end());

    return result;
}
};

```

26.4 Complexity

Time:

$O(m * n)$

Space:

$O(m * n)$

27. Full Problem Walkthrough 16 — Scramble String, Preview

27.1 Problem Summary

Determine if one string is a scrambled version of another.

27.2 State

$dp[len][i][j]$ = true if substring $s1$ starting at i with length len can scramble to substring $s2$ starting at j

27.3 Why This Is Advanced

This is 3D interval/string DP.

It uses split points inside substrings.

It will be revisited in interval DP / advanced string DP.

28. Full Problem Walkthrough 17 — Count Different Palindromic Subsequences, Preview

28.1 Problem Summary

Count distinct palindromic subsequences.

28.2 Why This Is Advanced

It combines:

- range DP
- duplicate handling
- counting
- modulo arithmetic

State:

$dp[l][r]$ = number of distinct palindromic subsequences in $s[l..r]$

But transitions are tricky because duplicates can cause overcounting.

This belongs in advanced DP practice.

29. Common String DP Edge Cases

29.1 Empty String

Important for:

- Edit Distance
- Regex Matching
- Wildcard Matching
- LCS

29.2 One Character

Important for palindrome DP.

29.3 Repeated Characters

Important for:

- Distinct Subsequences
- Palindromic Subsequences
- LCS

29.4 Zeros in Digit Strings

Important for Decode Ways.

29.5 Pattern Stars

Important for:

- Regex Matching
- Wildcard Matching

29.6 Substring vs Subsequence

Do not confuse.

30. Common String DP Mistakes

Mistake 1 — Off-by-One Indexing

If $dp[i][j]$ means first i and first j characters, use:

$s1[i - 1]$

$s2[j - 1]$

Mistake 2 — Wrong Empty Prefix Initialization

Edit distance needs:

dp[i][0] = i
dp[0][j] = j

Distinct subsequences needs:

dp[i][0] = 1

Mistake 3 — Confusing Substring and Subsequence

Longest Palindromic Substring is contiguous.

Longest Palindromic Subsequence is not.

Mistake 4 — Regex * Misunderstanding

In regex, * applies to previous character.

In wildcard, * stands alone and matches any sequence.

Mistake 5 — Wrong Range Fill Order

For dp[l][r], fill shorter lengths before longer lengths.

31. Interview Explanation Templates

31.1 LCS

I define $dp[i][j]$ as the LCS length of the first i characters of one string and first j characters of the other. If the current characters match, I extend the LCS by one. Otherwise, I skip one character from either string.

31.2 Edit Distance

I define $dp[i][j]$ as the minimum operations to convert prefixes. If characters match, no operation is needed. Otherwise, I consider insert, delete, and replace, and take the minimum.

31.3 Distinct Subsequences

For each character in s , if it matches the current character in t , I can either use it or skip it. If it does not match, I can only skip it.

31.4 Palindrome Range DP

I define $dp[l][r]$ for substring $s[l..r]$. Since palindrome structure depends on matching ends, the transition uses the inner substring $dp[l+1][r-1]$.

31.5 Word Break

I define $dp[i]$ as whether the prefix ending at i can be segmented. I try every split point and check if the left prefix is valid and the right substring is a dictionary word.

32. Complexity Patterns

LCS	$O(mn)$	$O(mn)$
Edit Distance	$O(mn)$	$O(mn)$

Problem	Time	Space
Delete Operation	$O(mn)$	$O(mn)$
Distinct Subsequences	$O(mn)$	$O(mn)$
Interleaving String	$O(mn)$	$O(mn)$
Regex Matching	$O(mn)$	$O(mn)$
Wildcard Matching	$O(mn)$	$O(mn)$
Longest Palindromic Subsequence	$O(n^2)$	$O(n^2)$
Longest Palindromic Substring	$O(n^2)$	$O(n^2)$
Palindromic Substrings	$O(n^2)$	$O(n^2)$
Palindrome Partitioning II	$O(n^2)$	$O(n^2)$
Word Break	$O(n^3)$ simple	$O(n)$
Decode Ways	$O(n)$	$O(n)$

33. Time-Boxed Practice Guidance

Intro String DP

Target:

25–40 minutes

Problems:

- LCS
- Delete Operation for Two Strings
- Decode Ways
- Word Break

Medium String DP

Target:

40–60 minutes

Problems:

- Edit Distance
- Interleaving String
- Longest Palindromic Subsequence
- Palindromic Substrings
- Minimum ASCII Delete Sum

Hard String DP

Target:

60–90 minutes

Problems:

- Regular Expression Matching
 - Wildcard Matching
 - Distinct Subsequences
 - Palindrome Partitioning II
 - Shortest Common Supersequence
-

34. Expanded Practice Set

34.1 Must-Do Medium

28. Longest Common Subsequence
29. Delete Operation for Two Strings
30. Edit Distance
31. Distinct Subsequences
32. Interleaving String
33. Word Break
34. Decode Ways
35. Minimum ASCII Delete Sum for Two Strings
36. Longest Palindromic Subsequence
37. Longest Palindromic Substring
38. Palindromic Substrings
39. Palindrome Partitioning II

34.2 Strong Interview Practice

40. Regular Expression Matching
41. Wildcard Matching
42. Shortest Common Supersequence
43. Minimum Insertion Steps to Make a String Palindrome
44. Longest String Chain
45. Count Palindromic Substrings
46. Count Different Palindromic Subsequences
47. Scramble String

34.3 Hard / Stretch

48. Regular Expression Matching
 49. Wildcard Matching
 50. Distinct Subsequences
 51. Scramble String
 52. Count Different Palindromic Subsequences
 53. Strange Printer
 54. Palindrome Partitioning III
 55. Encode String with Shortest Length
-

35. Suggested Study Order

Step 1 — Two-String Basics

Start with:

56. Longest Common Subsequence
57. Delete Operation for Two Strings
58. Minimum ASCII Delete Sum

Goal:

Understand $dp[i][j]$ prefix comparison.

Step 2 — Edit Operations

Solve:

- 59. Edit Distance
- 60. Interleaving String

Goal:

Learn insert/delete/replace and prefix states.

Step 3 — Counting Subsequences

Solve:

- 61. Distinct Subsequences

Goal:

Learn use-or-skip count transitions.

Step 4 — Palindrome DP

Solve:

- 62. Longest Palindromic Subsequence
- 63. Longest Palindromic Substring
- 64. Palindromic Substrings
- 65. Palindrome Partitioning II

Goal:

Learn range DP over substrings.

Step 5 — Pattern Matching

Solve:

- 66. Regular Expression Matching
- 67. Wildcard Matching

Goal:

Master $*$ transitions.

Step 6 — String Construction

Solve:

- 68. Shortest Common Supersequence

Goal:

Use DP table to reconstruct answer.

36. Pattern Decision Table

common subsequence	$dp[i][j]$	LCS
convert one string to another	$dp[i][j]$	Edit Distance
delete to make equal	LCS or $dp[i][j]$	Delete Operation
count subsequences	$dp[i][j]$ count	Distinct Subsequences
interleave two strings	$dp[i][j]$	Interleaving
regex with . and *	match prefixes	Regex Matching

Problem Clue	State	Example
wildcard with ? and *	match prefixes	Wildcard
palindromic subsequence	dp[l][r] length	LPS
palindromic substring	dp[l][r] boolean	LPS substring
segment string	dp[i] prefix valid	Word Break
decode digits	dp[i] count	Decode Ways
build shortest string	LCS reconstruction	SCS

37. Mini Problem 1 — Minimum Insertions to Make Palindrome

Key idea:

minimum insertions = n - longest palindromic subsequence

```

class Solution {
public:
    int minInsertions(string s) {
        int n = s.size();

        vector<vector<int>> dp(n, vector<int>(n, 0));

        for (int i = 0; i < n; i++) {
            dp[i][i] = 1;
        }

        for (int len = 2; len <= n; len++) {
            for (int l = 0; l + len - 1 < n; l++) {
                int r = l + len - 1;

                if (s[l] == s[r]) {
                    dp[l][r] = 2 + dp[l + 1][r - 1];
                } else {
                    dp[l][r] = max(dp[l + 1][r], dp[l][r - 1]);
                }
            }
        }

        return n - dp[0][n - 1];
    }
}

```

```
}  
};
```

38. Mini Problem 2 — Longest String Chain

Sort words by length.

```
class Solution {  
public:  
    bool isPredecessor(string& a, string& b) {  
        if (b.size() != a.size() + 1) {  
            return false;  
        }  
  
        int i = 0;  
        int j = 0;  
  
        while (j < b.size()) {  
            if (i < a.size() && a[i] == b[j]) {  
                i++;  
            }  
  
            j++;  
        }  
  
        return i == a.size();  
    }  
  
    int longestStrChain(vector<string>& words) {  
        sort(words.begin(), words.end(), [](const string& a, const string& b) {  
            return a.size() < b.size();  
        });  
  
        int n = words.size();  
  
        vector<int> dp(n, 1);  
        int answer = 1;  
  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < i; j++) {  
                if (isPredecessor(words[j], words[i])) {
```

```

        dp[i] = max(dp[i], dp[j] + 1);
    }
}

answer = max(answer, dp[i]);
}

return answer;
}
};

```

39. Mini Problem 3 — Space-Optimized LCS

```

class Solution {
public:
    int longestCommonSubsequence(string text1, string text2) {
        int m = text1.size();
        int n = text2.size();

        vector<int> prev(n + 1, 0);
        vector<int> curr(n + 1, 0);

        for (int i = 1; i <= m; i++) {
            fill(curr.begin(), curr.end(), 0);

            for (int j = 1; j <= n; j++) {
                if (text1[i - 1] == text2[j - 1]) {
                    curr[j] = 1 + prev[j - 1];
                } else {
                    curr[j] = max(prev[j], curr[j - 1]);
                }
            }

            prev = curr;
        }

        return prev[n];
    }
};

```

40. Mini Problem 4 — Expand Around Center Alternative for Palindromic Substrings

Not DP, but often better.

```
class Solution {
public:
    int expand(string& s, int left, int right) {
        int count = 0;

        while (left >= 0 && right < s.size() && s[left] == s[right]) {
            count++;
            left--;
            right++;
        }

        return count;
    }

    int countSubstrings(string s) {
        int answer = 0;

        for (int center = 0; center < s.size(); center++) {
            answer += expand(s, center, center);
            answer += expand(s, center, center + 1);
        }

        return answer;
    }
};
```

41. Common Edge Case Checklist

Before submitting string DP, test:

```
empty string
one character
two same characters
two different characters
all repeated characters
```

no matching characters
one string empty
both strings empty
pattern with only *
string starts with 0
string contains 10 / 20
palindrome length odd
palindrome length even

42. Debugging String DP

If string DP fails:

69. Define $dp[i][j]$ precisely.
70. Check if it means first i chars or index i .
71. Verify character access $s[i - 1]$.
72. Check empty prefix initialization.
73. Check match vs non-match transition.
74. Check substring vs subsequence.
75. Check fill order for $dp[l][r]$.
76. Test smallest examples manually.

43. What to Master Before Chapter 38

Before moving to Knapsack DP, make sure you can:

- Define prefix DP for two strings
- Solve LCS
- Solve Edit Distance
- Solve Delete Operation for Two Strings
- Solve Distinct Subsequences
- Solve Interleaving String
- Solve Word Break
- Solve Decode Ways
- Solve Longest Palindromic Subsequence
- Solve Palindromic Substrings
- Understand range DP on strings
- Handle empty prefixes
- Avoid substring/subsequence confusion
- Implement pattern matching DP basics

44. Final Cheat Sheet

LCS

```

if (a[i - 1] == b[j - 1]) {
    dp[i][j] = 1 + dp[i - 1][j - 1];
} else {
    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
}

```

Edit Distance

```

if (a[i - 1] == b[j - 1]) {
    dp[i][j] = dp[i - 1][j - 1];
} else {
    dp[i][j] = 1 + min({delete, insert, replace});
}

```

Distinct Subsequences

```

if (s[i - 1] == t[j - 1]) {
    dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];
} else {
    dp[i][j] = dp[i - 1][j];
}

```

Palindrome Subsequence

```

if (s[l] == s[r]) {
    dp[l][r] = 2 + dp[l + 1][r - 1];
} else {
    dp[l][r] = max(dp[l + 1][r], dp[l][r - 1]);
}

```

Palindrome Substring

```

dp[l][r] = s[l] == s[r] && (len <= 2 || dp[l + 1][r - 1]);

```

Word Break

```

dp[0] = true;

if (dp[j] && dict.count(s.substr(j, i - j))) {
    dp[i] = true;
}

```

45. Core Takeaway

String DP is mostly about prefixes and ranges.
For two strings, think:

```

dp[i][j] = answer for first i chars of string1 and first j chars of string2

```

For palindromes, think:

$dp[l][r]$ = answer for substring $s[l..r]$

The most important distinction is:

substring = contiguous

subsequence = can skip characters

Most string DP problems are built from:

- match characters
- skip characters
- insert/delete/replace
- use or skip current character
- shrink palindrome inward
- split string at a position

Once you can define the state clearly, string DP becomes much more systematic.